

Soccer Balls 2 (Web) — iOS gameplay performance post-mortem

TL;DR — iOS was re-uploading a full-screen terrain texture to the GPU *once per terrain object, per frame* (`texImage2D`), which stalls a tile-based-deferred mobile GPU. We now rasterize each static terrain texture **once** and only move its tile. Result: **15–25fps** → **a solid 60**.

Symptom

On iOS Safari (including “Add to Home Screen” standalone), in-game ran **10–25fps**, while the **pause menu held a clean 60**. Desktop and Android were always 60. Frame time was erratic — the on-screen `interval` counter swung **37–95ms** frame-to-frame.

What it *wasn't* (and the one number that mattered)

The usual prime suspect on iOS is `devicePixelRatio` **fill-rate**: sizing the drawing buffer to `cssPx × dpr` (2–3× on iPhones) means shading 4–9× the pixels. **Ruled out** — we ship `allow-high-dpi="false"`, so the WebGL backbuffer is a fixed **700×525** (confirmed on the perf HUD: `buf 700x525`). The retina upscale to the screen is done by the compositor and is cheap.

Also **ruled out** by reading the compiled WebGL context attributes:

`preserveDrawingBuffer: false` and `antialiasing: 0` — the other two classic iOS taxes were already off.

The decisive measurement was on the perf HUD:

Counter	Meaning	Reading in-game
<code>ourcode</code>	our JS per frame (physics + scene build)	~1 ms
<code>interval</code>	wall-clock per frame	37–95 ms

JS does essentially nothing, yet each frame takes 37–95ms. So **~all** the time is spent *outside JavaScript* — in OpenFL’s GPU render/present, which runs **after** our `MainLoop` returns (which is exactly why `ourcode` can’t see it). And because the `raf` counter reads a genuine low number (not a “lying 60”), we are **GPU-present-bound, not CPU-bound**.

The pause menu is fast for the same reason in reverse: `Game.Render()` early-returns when paused → nothing is re-rendered → nothing is re-uploaded → 60.

Root cause: a full-screen texture re-upload, per terrain object, per frame

Terrain is drawn by software-rasterizing each terrain object's vector fill into a `BitmapData`, then handing that bitmap to the GPU tile layer. In `GameObj.RenderPhysicsLineObject_Static` → `RenderFillAsTile`, every visible terrain object did this **every frame**:

```
lineFillBD.fillRect(...)           // clear a 700x525 bitmap
lineFillBD.draw(Game.fillScreenMC) // rasterize the shape
into it
TileRenderer.PushAt(lineFillBD, 0, 0) // hand it to the tilemap
```

In OpenFL, a `BitmapData` wraps an `Image` carrying a `version` counter. Any `draw()` / `fillRect()` / `copyPixels()` bumps `version`. The GL backend's texture-upload checks `textureVersion != image.version` and, if stale, re-uploads the entire bitmap via `gl.texImage2D`.

So redrawing the bitmap each frame ⇒ `version` bumps each frame ⇒ a fresh **full-screen CPU→GPU texture upload each frame** — and once *per terrain object*. With *N* visible terrain pieces, that is:

$N \times (700 \times 525 \times 4 \text{ bytes} \approx 1.4 \text{ MB})$ uploaded to the GPU, every frame.

That *N* is exactly why the frame time scaled **37 ↔ 95ms** with how much terrain was on screen.

Why only iOS

Apple GPUs are **tile-based deferred renderers (TBDR)**. They bin all the frame's geometry first, then shade per on-chip tile. A `texImage2D` into a texture that's referenced by the in-flight render forces the driver to either **flush/serialize the tile pipeline** or ghost (reallocate) the resource — and these CPU→GPU transfers don't overlap cleanly with rendering. Do that several times per frame and frames balloon to tens of milliseconds.

Desktop and Android use immediate-mode GPUs with mature streaming-upload paths and far more memory bandwidth, so per-frame texture re-uploads are nearly free there. **The same code path is invisible on desktop and fatal on iOS.** (We'd actually hit a

smaller version of this earlier with the perf-HUD `TextField` re-uploading every frame — same mechanism, smaller texture.)

The fix: rasterize once, reuse the texture, just move the tile

Terrain is **static** — the shape never changes, only the camera moves. So

`Settings.cachedTerrain` (default **ON**; `GameObj.BuildTerrainCache`) rasterizes each terrain object **once** into a bitmap sized to its world bounding box, then every frame just re-positions that cached tile:

```
TileRenderer.PushAt(cachedTerrainBD,  
    Math.round(cachedTerrainX - Game.camera.x),  
    Math.round(cachedTerrainY - Game.camera.y));
```

Because `cachedTerrainBD` is never redrawn, its `image.version` is **frozen**, so OpenFL uploads its GPU texture exactly once and reuses it thereafter (this rides the existing `TileRenderer.tilesetFor` cache, keyed by `BitmapData` identity). Per frame we now only mutate the tile's matrix — a couple of floats of vertex data, **no CPU→GPU transfer**. Zero per-frame `texImage2D`.

It's pixel-identical, not an approximation

The old path rasterized in **screen** space (points offset by `camera`) into a full-screen bitmap pushed at `(0,0)`. The cache rasterizes in **local** space (points offset by the bbox origin) and pushes the tile at `bbox0origin - camera`. Working through the `beginBitmapFill` matrix, **both the camera term and the bbox-origin term cancel out of the texture mapping** — the bitmap fill samples the same texel for the same world point in both cases. Output is identical modulo sub-pixel rounding.

Objects with a bounding box larger than 4096px fall back to the per-frame path automatically (a guard against pathological bitmap allocations).

Result & what's deliberately left

Verified on device: **solid 60**, and the 37 ↔ 95ms variance is gone (it was the N uploads).

The one remaining per-frame upload is the full-screen `screenBD` software underlay (static background + the aim-trajectory preview). A *single* upload per frame is cheap enough that we're at a clean 60, so it's left as-is. If it ever bites, the same trick applies: make the static background an upload-once layer and keep only genuinely dynamic pixels in a per-frame surface.

A “Cached terrain (perf)” toggle remains in Options to disable the cache for comparison; the temporary underlay/tiles A/B toggles used during diagnosis were removed (the `?nounderlay` / `?notiles` URL flags still work).

One-line version

iOS was re-uploading a full-screen terrain texture to the GPU once per terrain object per frame (`texImage2D` stalls a tile-based-deferred GPU); we now rasterize each static terrain texture once and only move its tile, so nothing re-uploads.

Key references in the code

- `src/GameObj.hx` — `RenderPhysicsLineObject_Static`, `RenderFillAsTile`, `BuildTerrainCache`, `cachedTerrainBD`
- `src/TileRenderer.hx` — `PushAt`, `tilesetFor` (BitmapData-identity Tileset cache)
- `src/Settings.hx` — `cachedTerrain` (default on)
- Perf HUD — `ourcode` (JS ms) vs `interval` (wall-clock ms); the fork that proved GPU-present-bound